Contents

1 Changelog: Version 1 to Version 2 **2**

2 Description of Keltner Channel Trading Strategy **2**

3 History and Online resources for the Strategy **3**

4 Publisher's claim on strategy's performance **3**

5 My findings on the Strategy's performance **4**

6 Performance Graphs **4**

7 Backtesting assumptions **5**

8 Optimization for Keltner Strategy **5**

9 Performance on currencies **8**

9.1 EURUSD=X 8

9.2 GBPUSD=X 9

9.3 JPY=X 9

9.4 AUDUSD=X 10

9.5 CAD=X 11

10 Appendix **11**

Quantitative Financial Analytics and Algorithmic Trading
Keltner Channel Trading Strategy
Initial Backtest Report

Sakshi Singh

September 30, 2024

1 Changelog: Version 1 to Version 2

Version	Updates
Version 1	Initial implementation of the Keltner Channel strategy using a manually created framework.
Version 2	Updated references as citations instead of links Updated starting price to be same as the article Replicated the backtest in article using backtesting.py Updated the equity curve with plot generated from backtesting.py Created an optimization universe using backtesting.py
Version 3	Updated the optimization code in backtesting.py Added table of content Ran the strategy on 5 currencies

Table 1: Summary of Updates from Version 1 to Version 2

2 Description of Keltner Channel Trading Strategy

Keltner Channels make use of volatility in the market price of a stock by utilizing the Average True Range (ATR) to determine the distance of the upper and lower bands from a central moving average[1]. The bands expand and contract based on changes in volatility: when the market is more volatile, the bands widen, and when volatility decreases, the bands contract. The strategy can be implemented with the below formulation:

```
data['ATR'] = talib.ATR(data['High'], data['Low'], data['Close'],
                        timeperiod=period)
```

[1]Multiple Moving averages can be used, like Simple Moving Average, or Exponential Moving Average. There was a confusion about which Moving Average was used in the article which I am replicating. In one place its written EMA, which during formulation, it's just MA. I was able to achieve similar results as them using MA, hence I have used Simple Moving Average (MA) for now.

```
data['middle'] = talib.MA((data['Close'] + data['Low'] + data['High'])/3,
                          timeperiod=self.period)

data['upper'] = data['middle'] + atr_factor*data['ATR']
data['lower'] = data['middle'] - atr_factor*data['ATR']
```

Once these lower and upper bands are created, below logic can be used to decide when to buy and sell[2].

```
if prev_close < prev_lower :
    # if cash is available
    buy(stock)
elif prev_close > prev_middle:
    # if stock is available to sell
    sell(stock)
```

3 History and Online resources for the Strategy

The Keltner Channels were introduced by Chester W. Keltner in 1960, using a 10-day simple moving average (SMA) of the typical price (known as middle in my code above) to identify trends, with the channel width based on the 10-day SMA of the trading range. In the 1980s, Linda Raschke replaced the band range with the Average True Range (ATR) to measure volatility, instead of using SMA, and Robert Colby later suggested using an exponential moving average (EMA).
Despite its effectiveness, Keltner Channels were overshadowed by the Bollinger Bands, introduced by John Bollinger in the 1980s, which also used volatility (standard deviation) but became more popular due to their simplicity and flexibility in various market conditions. Bollinger Bands' broad adoption made them a preferred tool for traders, limiting the visibility of Keltner Channels.
Initially, the Keltner Channel was primarily applied to commodities markets, as Chester Keltner's work focused on these assets. Over time, Keltner Channels have been applied to the stock market as well by signaling potential breakouts or breakdowns, traders can buy or sell at more opportune moments. Forex traders can also use Keltner Channels to assess volatility and gauge potential entry and exit points.

4 Publisher's claim on strategy's performance

I have mainly referred to the article by Quantified Strategies [3].
According to the original publisher, the Keltner Channel strategy demonstrates a notable win rate of around 80%, reflecting its effectiveness in identifying trends and potential trading opportunities across various financial markets. They talk about leveraging the Keltner

[2] It was unclear from the article if previous day's or current day's lower/upper/middle bands has to be used. But since calculating bands require current day's High, Low and Close, which we won't have available, I am using previous day's bands

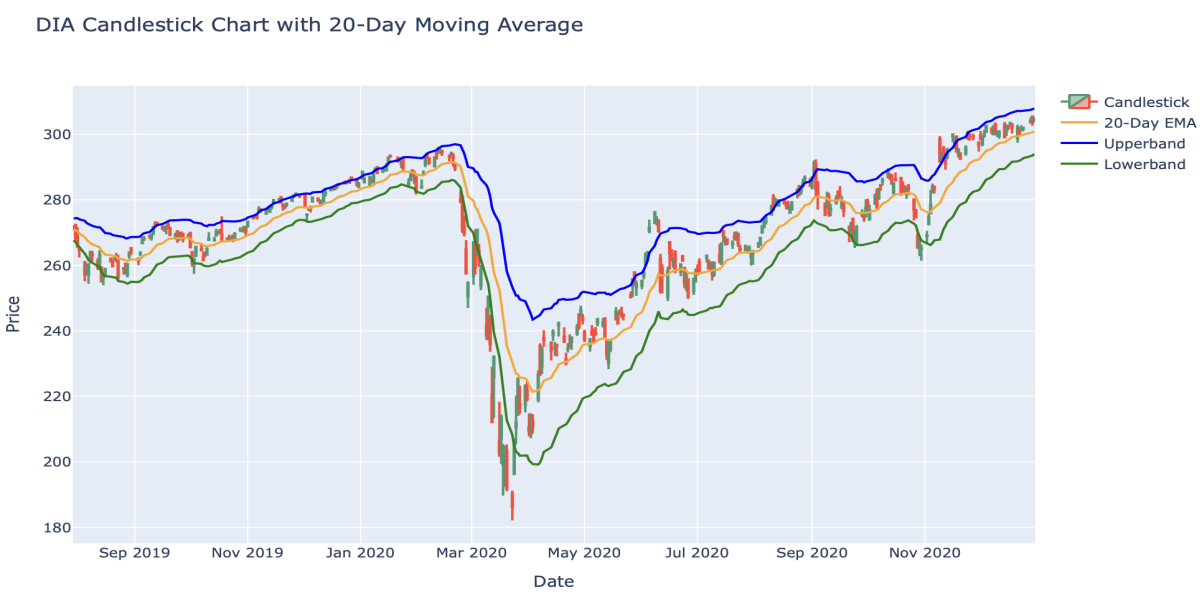
Channel's ability to provide clear signals for entry and exit points.
Win Rate $= \frac{\text{Number of Winning Trades}}{\text{Total Number of Trades}} \times 100\%$.
This high win rate suggests that the strategy is successful in approximately 80% of trades. In backtests, the strategy has shown strong performance with a six-day period and an ATR of 1.3 on the S&P 500, yielding 205 trades with a 6.14% CAGR, a 12.6% time in market, and a profit factor of 2.08. However, its performance has been less favorable since 2016, showing minimal returns.
Using a longer period, such as 30 days with an ATR of 1.1, resulted in 158 trades and a 4.7% CAGR on the S&P 500. This backtest is what I have tried to replicate on my own as well.
The author also mentioned that while the Keltner Channel strategy can be profitable, its effectiveness varies with market conditions, and incorporating other technical indicators or adjusting parameters may improve results.

5 My findings on the Strategy's performance

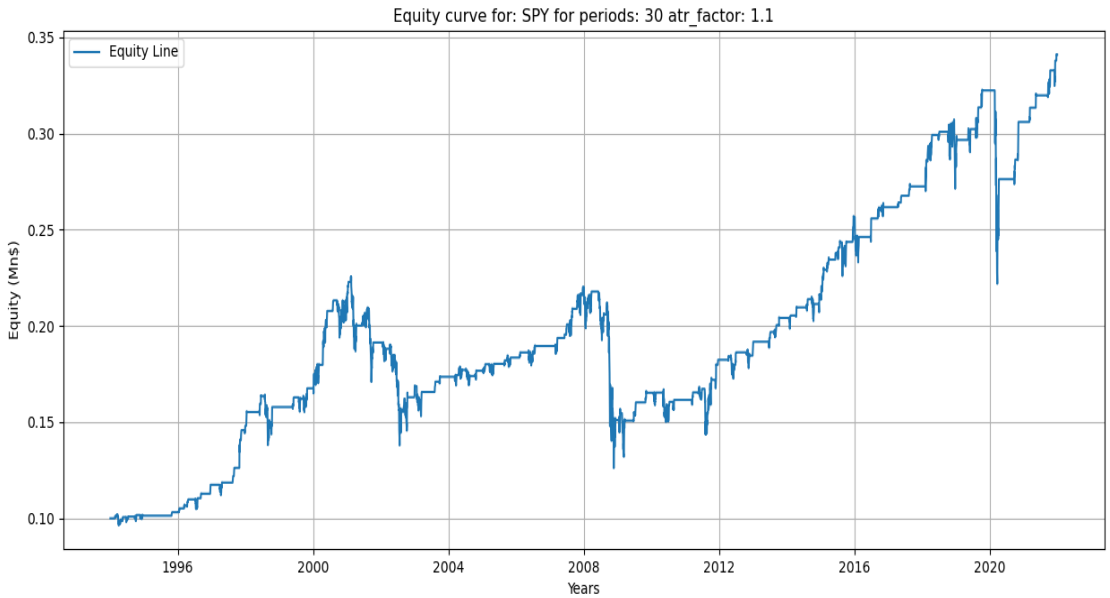
In my attempt to replicate the Keltner Channel strategy analysis, I focused on the SPY ETF. Using a period of 30 days and an ATR of 1.1, I was able to achieve a total of 160 trades. The resulting compound annual growth rate (CAGR) was 4.67%. Although I was unable to replicate the entire plot as originally presented, these results provide insight into the performance of the strategy with the specified parameters.
I replicated the backtesting using the bakctesting.py framework, and was able to get the 80% win rate as mentioned in the article with SPY.

6 Performance Graphs

I first replicated the depiction of keltner bands with Dow Jones Index (DIA), periods of 20 days and ATR factor of 2. Below is the plot:



Running the backtest on SPY, with period = 30 and atr_factor = 1.1, below is the equity curve I was able to get through my code for backtesting.



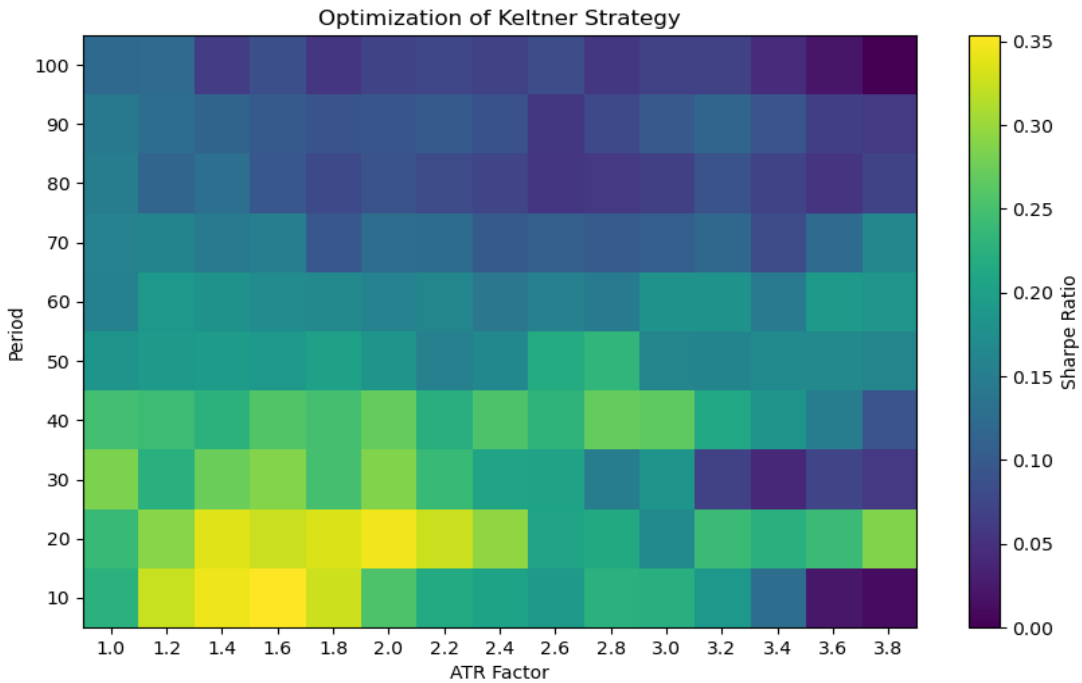
I get a total of 160 trades, with a CGAR of 4.67%, profit factor of 1.95, and win rate of 80%.

7 Backtesting assumptions

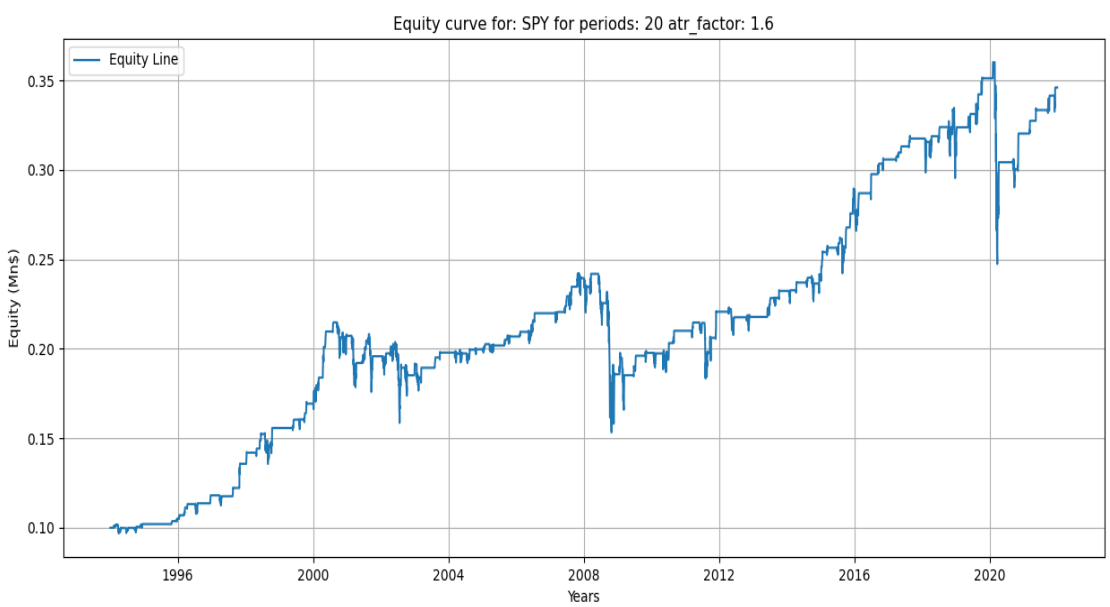
The data has been downloaded from yahoo finance module: yfinance.
Also, instead of EMA, I have taken SMA or MA for calculating the mid line. It was a little unclear on what the original authors have used.
I have assumed 0 commission or trading cost for this analysis.
I backested the data starting from 1994, from what I could infer from the backtest plots on the article.

8 Optimization for Keltner Strategy

To improve the Keltner Channel strategy's performance, I optimized two key parameters: the moving average period and the ATR factor. I implemented the `optimize_keltner_strategy` function, which evaluates different combinations of these parameters to maximize the Sharpe Ratio. This optimization can be done on Profit Factor, Win rate etc as well. A commission of 0.2% was added to each trade to account for realistic trading conditions. I chose atr_rate and periods as hyperparameters to optimize over, since these are the two hyperparameters used in the article as well.
Using a backtesting framework, I iterated over a range of periods and ATR factors. The results were visualized in a heatmap, showing the parameter combinations that produced the highest sharpe ratio. This approach helped identify the optimal settings for maximizing strategy profitability. Below is the heatmap for optimization of , at 0.2% commission:



The highest win rate is achieved at lower end of periods and around 1.2-2 atr factor. I would want to concentrate myself on a region that's stable, ie, where the results doesn't change drastically with slight variation in parameters. So I would ideally want to run the strategy at period = 20 and atr_factor = 1.6. Below is the equity curve gebnerated with this configuration:



This equity curve is similar to before, profitable, and also accounts for commission, hence closer to real world scenario.

Below is the code for optimization framework. I first optimize the strategy using backtest.py's optimization functionality. Then I create a heatmap around the best parameters I receive, to check the stability of the output:

```
def optimize_keltner_strategy(data, KeltnerStrategy, periods, atr_factors, metric):
    bt = Backtest(data, KeltnerStrategy, cash=10**5, commission=0.002)
    KeltnerStrategy.data = data

    # Optimize the parameters
    stats = bt.optimize(period=periods, atr_factor=atr_factors, maximize=metric)

    return stats

cash = 10**5
metric = "Sharpe Ratio"
periods = range(10, 101, 10)
atr_factors = list(np.arange(1, 4, 0.2))

stats = km.optimize_keltner_strategy(data, km.KeltnerStrategy, periods,
atr_factors, metric)

print("details on best strategy: ")
print("period: ", stats['_strategy'].period, "atr_factor: ",
stats['_strategy'].atr_factor)

days = stats['Duration'].days
years = days / 365.25

# CAGR calculation
cagr = (stats["Equity Final [$]"] / cash) ** (1 / years) - 1
cagr_percentage = cagr * 100

print("CGAR [%]: ", cagr_percentage)
print("Win Rate [%]: ", stats["Win Rate [%]"])
print("Profit Factor: ", stats["Profit Factor"])
print("Return [%]: ", stats["Return [%]"])
print("Equity Final [$]: ", stats["Equity Final [$]"])
print("# Trades: ", stats["# Trades"])

# create a heatmap around the best output
params = {'period':  stats['_strategy'].period, 'atr_factor':
stats['_strategy'].atr_factor}
params['cash'] = cash
performance = km.make_heatmap_array_around_params(data, km.KeltnerStrategy,
params, metric)
```

```
plt.figure(figsize=(10, 6))
plt.imshow(performance, cmap='viridis', aspect='auto', origin='lower')
plt.colorbar(label=metric)
plt.xlabel('Period')
plt.ylabel('ATR Factor')
plt.title('Optimization of Keltner Strategy')
plt.show()
```

9 Performance on currencies

In my report, I pulled the last 1.5 years of data using Yahoo Finance for the following five currency pairs: ["EURUSD=X", "GBPUSD=X", "JPY=X", "AUDUSD=X", "CAD=X"]. The historical data includes open, high, low, close information, which I used to analyze and backtest trading strategies. This data is pulled at 1hr frequency. Below is the code to pull this data:

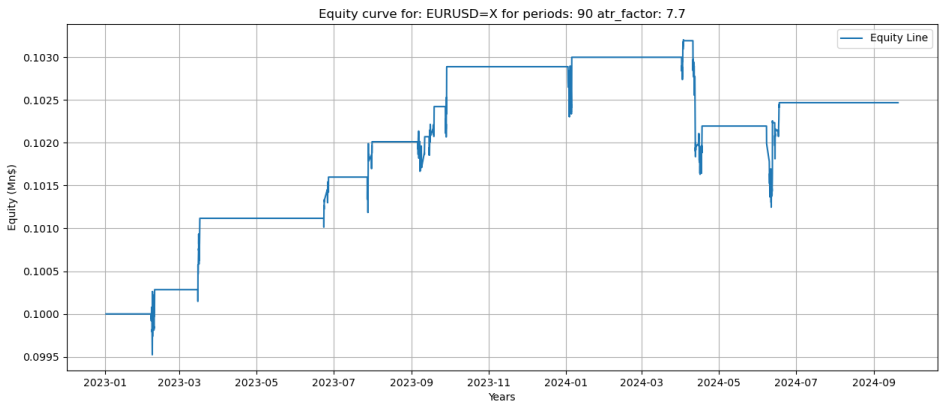
```
tickers = ["EURUSD=X", "GBPUSD=X", "JPY=X", "AUDUSD=X", "CAD=X"]
start_date = "2023-01-01"
end_date = "2024-09-20"

currency_df = dict()

# Loop through each ticker and download 1-hour data
for ticker in tickers:
    data = yf.download(ticker, start=start_date, end=end_date, interval='1h')

    # Drop 'Volume' column if it exists
    if 'Volume' in data.columns:
        data = data.drop(columns=['Volume'])

    currency_df[ticker] = data
```

Then I ran the optimization on data for each of these currencies.
I optimized my strategies using `backtesting.py`'s optimization functionality. The parameters optimized included factors such as moving average periods, ATR factors. Starting cash is set to 10^5\$. After running the optimizations on all five currency pairs, I obtained the following results:

9.1 EURUSD=X

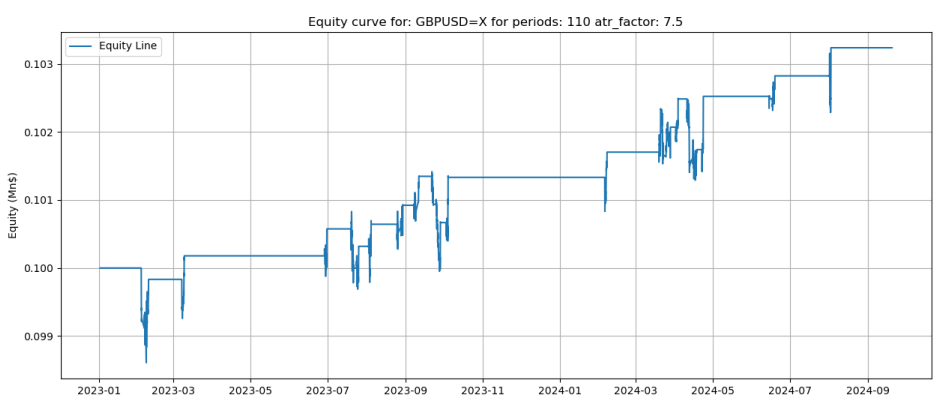
```
details on best strategy for:  EURUSD=X
period:  90 atr_factor:  7.7
CGAR [%]:  1.43
```

```
Win Rate [%]:  91.67
Profit Factor:  3.54
Return [%]:  2.47
Equity Final [$]:  102468.91
# Trades:  12
```



9.2 GBPUSD=X

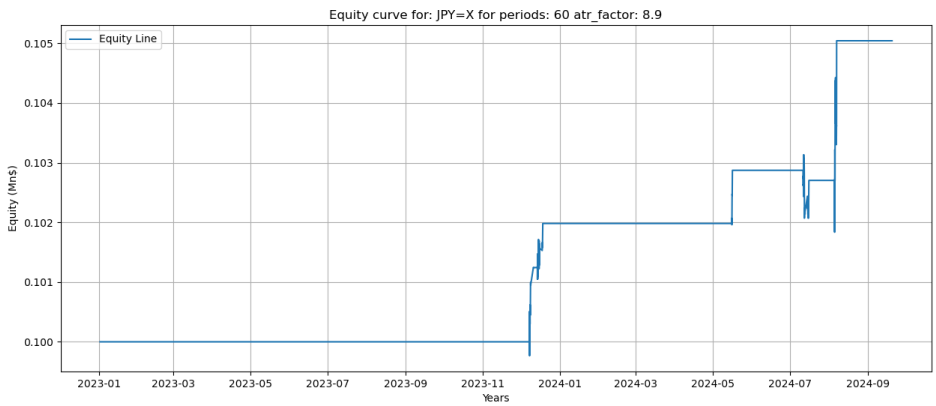
```
details on best strategy for:  GBPUSD=X
period:  110 atr_factor:  7.5
CGAR [%]:  1.88
Win Rate [%]:  70.59
Profit Factor:  2.55
Return [%]:  3.24
Equity Final [$]:  103237.81
# Trades:  17
```



9.3 JPY=X

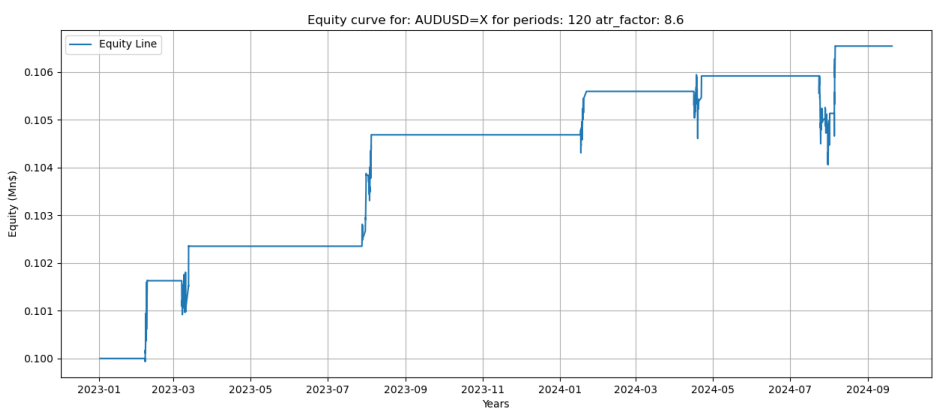
```
details on best strategy for:  JPY=X
```

```
period:  60 atr_factor:  8.9
CGAR [%]:  2.91
Win Rate [%]:  80.0
Profit Factor:  31.19
Return [%]:  5.04
Equity Final [$]:  105040.8
# Trades:  5
```



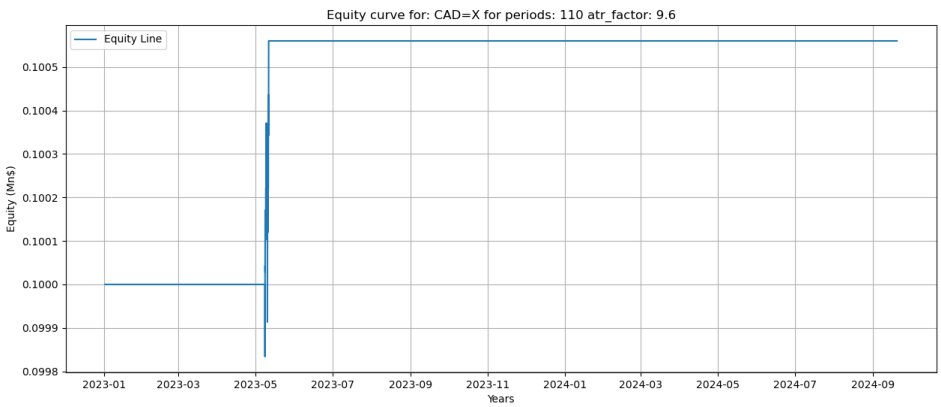
9.4 AUDUSD=X

```
details on best strategy for:  AUDUSD=X
period:  120 atr_factor:  8.6
CGAR [%]:  3.77
Win Rate [%]:  87.5
Profit Factor:  9.63
Return [%]:  6.54
Equity Final [$]:  106540.17
# Trades:  8
```



9.5 CAD=X

```
details on best strategy for:  CAD=X
period:  110 atr_factor:  9.6
CGAR [%]:  0.33
Win Rate [%]:  100.0
Profit Factor:  nan
Return [%]:  0.56
Equity Final [$]:  100559.87
# Trades:  1
```



References

[1] Lite Finance. *Best Technical Indicators: Keltner Channel*. https://www.litefinance.org/blog/for-beginners/best-technical-indicators/keltner-channel/

[2] The Robust Trader. *Keltner Channel*. https://therobusttrader.com/keltner-channel/

[3] Quantified Strategies. *Keltner Bands Trading Strategies*. https://www.quantifiedstrategies.com/keltner-bands-trading-strategies/

[4] How to Trade. *Keltner Channels*. https://howtotrade.com/trading-strategies/keltner-channels/

10 Appendix

Appendix - Code used to replicate article's backtest

```
import pandas as pd
import yfinance as yf
import talib
```

```
# to plot the candlestick dataset, with Keltner bands on Dow Jones Index
start_date = "2019-07-01"
end_date = "2020-12-31"

ticker = "DIA"
data = yf.download(ticker, start=start_date, end=end_date)
data = data.drop(columns=['Volume'])

period = 20

data['ATR'] = talib.ATR(data['High'], data['Low'], data['Close'], timeperiod=period)

data['EMA_20'] = talib.EMA(data['Close'], timeperiod=period)

data['upperband'] = data['EMA_20'] + 2*data['ATR']
data['lowerband'] = data['EMA_20'] - 2*data['ATR']

data.dropna(inplace=True)

import plotly.graph_objects as go

fig = go.Figure()

fig.add_trace(go.Candlestick(x=data.index, open=data['Open'], high=data['High'],
low=data['Low'], close=data['Close'], name='Candlestick'))

fig.add_trace(go.Scatter(x=data.index, y=data['EMA_20'], mode='lines',
name='20-Day EMA', line=dict(color='orange')))
fig.add_trace(go.Scatter(x=data.index, y=data['upperband'], mode='lines',
name='Upperband', line=dict(color='blue')))
fig.add_trace(go.Scatter(x=data.index, y=data['lowerband'], mode='lines',
name='Lowerband', line=dict(color='green')))

fig.update_layout(title=f'{ticker} Candlestick Chart with 20-Day Moving Average',
xaxis_title='Date', yaxis_title='Price', xaxis_rangeslider_visible=False,
width=1000, height=600)

fig.show()


# To plot the equity curve for SPY
class strategyBacktest:
    def __init__(self, data, period, atr_factor, startdate, cash, stock):
        self.orig_data = data
        self.portfolio = {}
```

```
        self.period = period
        self.atr_factor = atr_factor
        self.cash = cash
        self.curr_date = startdate
        self.stock = stock
        self.close = 0
        self.middle = 0
        self.lower = 0
        self.upper = 0
        self.open = 0
        self.equity_curve = {}
        self.buy_trades = []
        self.sell_trades = []
        self.buy_price = 0
        self.sell_price = 0

    def view_portfolio(self):
        print(self.portfolio)

    def update_date(self):
        next_day = self.curr_date + timedelta(1)
        if next_day.weekday() == 5:  # Saturday
            next_day += timedelta(days=2)
        elif next_day.weekday() == 6:  # Sunday
            next_day += timedelta(days=1)
        self.curr_date = next_day

    def prepare_keltner_data(self, startdate):
        data = self.orig_data
        data = data.loc[startdate:]

        data['ATR'] = talib.ATR(data['High'], data['Low'], data['Close'],
        timeperiod=self.period)

        data['middle'] = talib.MA((data['Close'] + data['Low'] + data['High'])/3,
        timeperiod=self.period)
        data['upper'] = data['middle'] + self.atr_factor*data['ATR']
        data['lower'] = data['middle'] - self.atr_factor*data['ATR']

        data.dropna(inplace=True)

        self.data = data
        self.curr_date = data.index.min()

    def update_prices(self):
```

```
        self.prev_date = self.curr_date
        self.prev_close = self.close
        self.prev_middle = self.middle
        self.prev_lower = self.lower
        self.prev_upper = self.upper
        self.prev_open = self.open

        curr_date = self.curr_date
        self.close = self.data['Close'][curr_date]
        self.middle = self.data['middle'][curr_date]
        self.lower = self.data['lower'][curr_date]
        self.upper = self.data['upper'][curr_date]
        self.open = self.data['Open'][curr_date]

    def sell(self, stock, quantity=None, cash=None):
        if stock not in self.portfolio.keys():
            raise KeyError(stock, " not available to sell")

        if self.portfolio[stock]['quantity'] <= 0:
            raise KeyError(stock, " not available to sell")

        if ((not quantity) & (not cash)):
            quantity = self.portfolio[stock]['quantity']

        if cash:
            quantity = cash/self.open

        if self.portfolio[stock]['quantity'] < quantity:
            quantity = min(quantity, self.portfolio[stock]['quantity'])
            print("can only sell upto available: ", quantity)

        self.portfolio[stock]['quantity'] -= quantity

        self.cash = self.cash + quantity*self.open
        self.sell_trades.append(self.curr_date)
        print("sold")

    def buy(self, stock, quantity=None, cash=None):
        if ((not quantity) & (not cash)):
            cash = self.cash
            if cash == 0:
                raise BaseException("No cash to buy")

        if cash:
            quantity = cash/self.open
```

```
        if stock not in self.portfolio.keys():
            self.portfolio[stock] = {'quantity': 0}

        self.portfolio[stock]['quantity'] += quantity
        self.cash = self.cash - quantity*self.open
        self.buy_trades.append(self.curr_date)
        print("bought")

    def next(self):
        if self.prev_close < self.prev_lower :
            if self.cash > 0:
                self.buy(self.stock)

        elif self.prev_close > self.prev_middle:
            if self.stock in self.portfolio.keys():
                if self.portfolio[self.stock]['quantity'] > 0:
                    self.sell(self.stock)

        self.equity_curve[self.curr_date] = self.cash
        if self.stock in self.portfolio.keys():
            self.equity_curve[self.curr_date] =
            self.portfolio[self.stock]['quantity']*self.close + self.cash

        self.update_date()
        self.update_prices()


start_date = "1994-01-01"
end_date = "2020-12-31"
ticker = "SPY"
data = yf.download(ticker, start=start_date, end=end_date)
data = data.drop(columns=['Volume'])

all_dates = pd.date_range(start=start_date, end=end_date, freq='B')
data = data.reindex(all_dates)
data = data.ffill()

bt = strategyBacktest(data, 30, 1.1, datetime.strptime(start_date, "%Y-%m-%d"),
10000, ticker)
bt.prepare_keltner_data(datetime.strptime(start_date, "%Y-%m-%d"))
bt.update_date()
bt.update_prices()
bt.update_date()
bt.update_prices()
```

```
for i in range(1000):
    bt.next()

print(len(bt.buy_trades))

equity_curve = pd.DataFrame(bt.equity_curve, index=[0]).T
equity_curve.columns = ["ticker = SPY, period = 30 days, atr factor = 1.1"]
equity_curve.plot(figsize=(12,6), xlabel='date', ylabel = 'Portfolio')

years = ((bt.sell_trades[-1] - bt.buy_trades[0])/365).days

cgar = (equity_curve.iloc[-1]/equity_curve.iloc[0])**(1/years) - 1
cgar
```